

LAPORAN TUGAS BESAR
IF2211 STRATEGI ALGORITMA
Tugas Besar 3 Strategi Algoritma



Kelompok 50: gurt:yo

Nathan Jovial Hartono	13523032
-----------------------	----------

Sebastian Hung Yansen	13523070
-----------------------	----------

Karol Yangqian Poetrachya	13523093
---------------------------	----------

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132

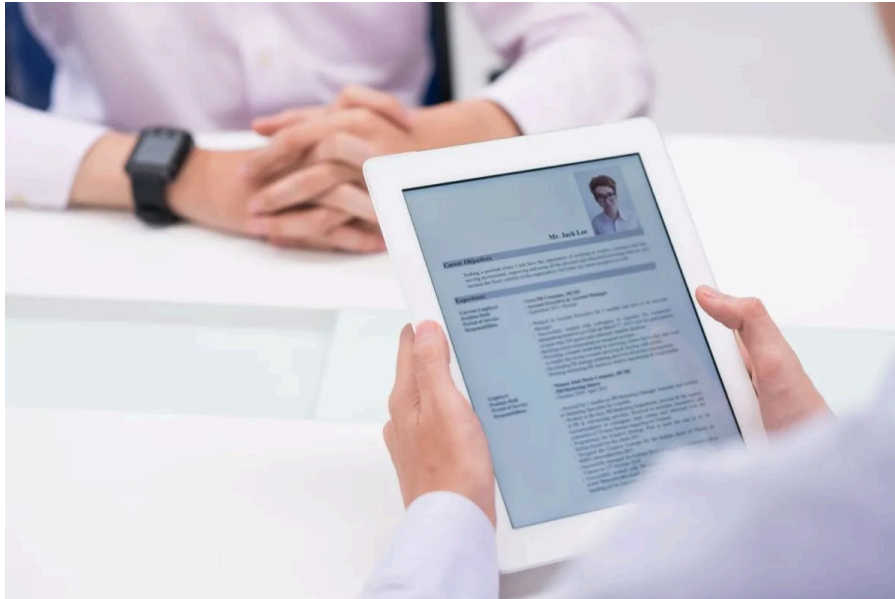
2025

DAFTAR ISI

DAFTAR ISI.....	1
BAB I DESKRIPSI TUGAS.....	2
BAB II LANDASAN TEORI.....	4
2.1. Pattern Matching.....	4
2.2. Algoritma Knuth-Morris-Pratt.....	4
2.3. Algoritma Boyer-Moore.....	5
2.4. Algoritma Fuzzy.....	5
2.5. Algoritma Aho-Corasick.....	6
2.6. Penjelasan Framework Aplikasi.....	8
BAB III ANALISIS PEMECAHAN MASALAH.....	9
3.1. Langkah-Langkah Pemecahan Masalah.....	9
3.2. Proses Pemetaan Masalah.....	9
3.3. Arsitektur Aplikasi.....	9
3.4. Alur Kerja Aplikasi.....	11
BAB IV IMPLEMENTASI DAN PENGUJIAN.....	12
4.1. Spesifikasi Teknis Program.....	12
4.1.1. Struktur Repository.....	12
4.1.2. Struktur Data.....	13
4.1.3. Fungsi dan Prosedur.....	14
4.2. Cara Menggunakan Program.....	22
4.3. Hasil Pengujian.....	22
BAB V KESIMPULAN DAN SARAN.....	33
5.1. Kesimpulan.....	33
5.2. Saran.....	33
5.3. Refleksi.....	33
LAMPIRAN.....	35
DAFTAR PUSTAKA.....	36

BAB I

DESKRIPSI TUGAS



Gambar 1. CV ATS dalam Dunia Kerja

(Sumber: <https://www.antaraneews.com/>)

Di era digital ini, keamanan data dan akses menjadi semakin penting. Perkembangan proses rekrutmen tenaga kerja telah mengalami perubahan signifikan dengan memanfaatkan teknologi untuk meningkatkan efisiensi dan akurasi. Salah satu inovasi yang menjadi solusi utama adalah Applicant Tracking System (ATS), yang dirancang untuk mempermudah perusahaan dalam menyaring dan mencocokkan informasi kandidat dari berkas lamaran, khususnya Curriculum Vitae (CV). ATS memungkinkan perusahaan untuk mengelola ribuan dokumen lamaran secara otomatis dan memastikan kandidat yang relevan dapat ditemukan dengan cepat.

Meskipun demikian, salah satu tantangan besar dalam pengembangan sistem ATS adalah kemampuan untuk memproses dokumen CV dalam format PDF yang tidak selalu terstruktur. Dokumen seperti ini memerlukan metode canggih untuk mengekstrak informasi penting seperti identitas, pengalaman kerja, keahlian, dan riwayat pendidikan secara efisien. Pattern matching menjadi solusi ideal dalam menghadapi tantangan ini.

Pattern matching adalah teknik untuk menemukan dan mencocokkan pola tertentu dalam teks. Dalam konteks ini, algoritma Boyer-Moore dan Knuth-Morris-Pratt (KMP)

sering digunakan karena keduanya menawarkan efisiensi tinggi untuk pencarian teks di dokumen besar. Algoritma ini memungkinkan sistem ATS untuk mengidentifikasi informasi penting dari CV pelamar dengan kecepatan dan akurasi yang optimal.

Di dalam Tugas Besar 3 ini, kami diminta untuk mengimplementasikan sistem yang dapat melakukan deteksi informasi pelamar berbasis dokumen CV digital. Metode yang akan digunakan untuk melakukan deteksi pola dalam CV adalah Boyer-Moore dan Knuth-Morris-Pratt. Selain itu, sistem ini akan dihubungkan dengan identitas kandidat melalui basis data sehingga harapannya terbentuk sebuah sistem yang dapat mengenali profil pelamar secara lengkap hanya dengan menggunakan CV digital.

BAB II

LANDASAN TEORI

2.1. Pattern Matching

Pattern matching merupakan proses yang bertujuan untuk menemukan dan mencocokkan pola tertentu dalam suatu teks. Secara formal, pola didefinisikan sebagai string dengan panjang m karakter yang akan dicari dalam teks berupa string dengan panjang n , di mana $m \ll n$. Dalam konteks pemrosesan string, terdapat dua pendekatan utama dalam pattern matching, yaitu pencocokan eksak (exact matching) dan pencocokan kabur (fuzzy matching).

Pencocokan eksak mengharuskan kesesuaian sempurna antara pola pencarian dengan substring dalam teks target, tanpa toleransi terhadap perbedaan karakter. Dua algoritma klasik yang telah terbukti efektif dalam pencocokan eksak adalah algoritma Knuth-Morris-Pratt (KMP) dan algoritma Boyer-Moore, yang masing-masing memiliki strategi optimisasi berbeda untuk meminimalkan jumlah perbandingan karakter. Sebaliknya, fuzzy matching memberikan fleksibilitas dengan memungkinkan pencarian pola yang memiliki kemiripan dengan teks target meskipun tidak identik secara karakter, sehingga dapat mengakomodasi variasi ejaan, kesalahan pengetikan, atau perbedaan format yang umum terjadi dalam data teks dalam kasus nyata.

2.2. Algoritma Knuth-Morris-Pratt

Algoritma Knuth-Morris-Pratt (KMP) mencari pola dalam teks dengan urutan dari kiri ke kanan. Ditemukan oleh Donald E. Knuth, James H. Morris, Jr., and Vaughan R. Pratt, algoritma ini menggeser pola dengan lebih cerdas saat proses pencarian dalam teks, menjadikannya lebih efisien daripada pendekatan *brute force* dan dapat memperoleh jumlah operasi pencocokan yang lebih sedikit dengan menghindari perbandingan yang tidak perlu.

Prinsip kerja algoritma KMP adalah ketika terjadi ketidakcocokan pada posisi tertentu dalam pola, algoritma tidak perlu memulai pencarian dari awal, melainkan dapat melompat ke posisi yang optimal berdasarkan struktur pola itu sendiri. Algoritma ini menerapkan prapemrosesan pola yakni membentuk tabel fungsi

pinggiran (*border function*) yang dinotasikan sebagai $b(k)$. Tabel ini menyatakan panjang prefiks terpanjang pada pola dari indeks 0 ke k yang juga merupakan sufiks pola dari indeks 1 hingga k , dengan k adalah satu sebelum indeks karakter terjadinya ketidakcocokan saat proses perbandingan string. Ketika terjadi ketidakcocokan pola pada indeks j dengan teks pada indeks i , pencocokan dilakukan lagi di indeks $b(j-1)$ pada pola. Jika ketidakcocokan terjadi pada karakter pertama pola, maka pencocokan berikutnya dilakukan terhadap karakter teks di posisi $i + 1$.

2.3. Algoritma Boyer-Moore

Algoritma Boyer-Moore melakukan perbandingan dengan mengiterasi mundur pada pola dimulai dari karakter terakhirnya. Teknik ini dinamakan *looking-glass technique*. Ketika terjadi ketidakcocokan karakter pola pada indeks j (dari belakang) dengan karakter teks pada indeks i (dari depan), terdapat tiga kemungkinan:

1. Jika karakter $T[i]$ pada teks terkandung dalam pola setelah posisi j (sebelah kiri), pola digeser ke kanan untuk membuat kemunculan terakhir karakter tersebut pada pola dengan $T[i]$.
2. Jika karakter $T[i]$ terdapat pada pola sebelum indeks j (sebelah kanan), pola digeser ke kanan satu kali.
3. Selain itu, pola digeser untuk membuat karakter $P[0]$ pada pola sejajar dengan $T[i+1]$.

Algoritma ini melakukan prapemrosesan pola dengan alfabet untuk membangun fungsi *last occurrence* $L()$. Bila x adalah sebuah huruf dalam alfabet, $L(x)$ didefinisikan sebagai index terbesar i sehingga karakter $P[i]$ pada pola sama dengan x atau mengembalikan -1 jika tidak ada indeks yang memenuhi syarat tersebut. Fungsi ini berupa tabel yang dikalkulasi ketika pola sedang dibaca.

2.4. Algoritma Fuzzy

Algoritma Fuzzy digunakan untuk melakukan *pattern matching* yang tidak eksak. Pola dibandingkan terhadap segmen isi teks dengan menghitung kemiripannya berdasarkan *Levenshtein Distance*. Metrik ini mengukur berapa jumlah operasi penghapusan, penyisipan, atau substitusi yang diperlukan terhadap segmen teks untuk mendapatkan string yang sama dengan pola yang diberikan. Semakin besar kecil *distance* ini, maka dua string yang dibandingkan semakin mirip. Nilai 0 menyatakan

bahwa kedua string sama persis. Dalam penerapannya, batas atas untuk *Levenshtein Distance* ditetapkan untuk menyatakan bahwa dua string adalah mirip.

$$\text{lev}_{a,b}(i, j) = \begin{cases} \max(i, j) & \text{if } \min(i, j) = 0, \\ \min \begin{cases} \text{lev}_{a,b}(i-1, j) + 1 \\ \text{lev}_{a,b}(i, j-1) + 1 \\ \text{lev}_{a,b}(i-1, j-1) + 1_{(a_i \neq b_j)} \end{cases} & \text{otherwise.} \end{cases}$$

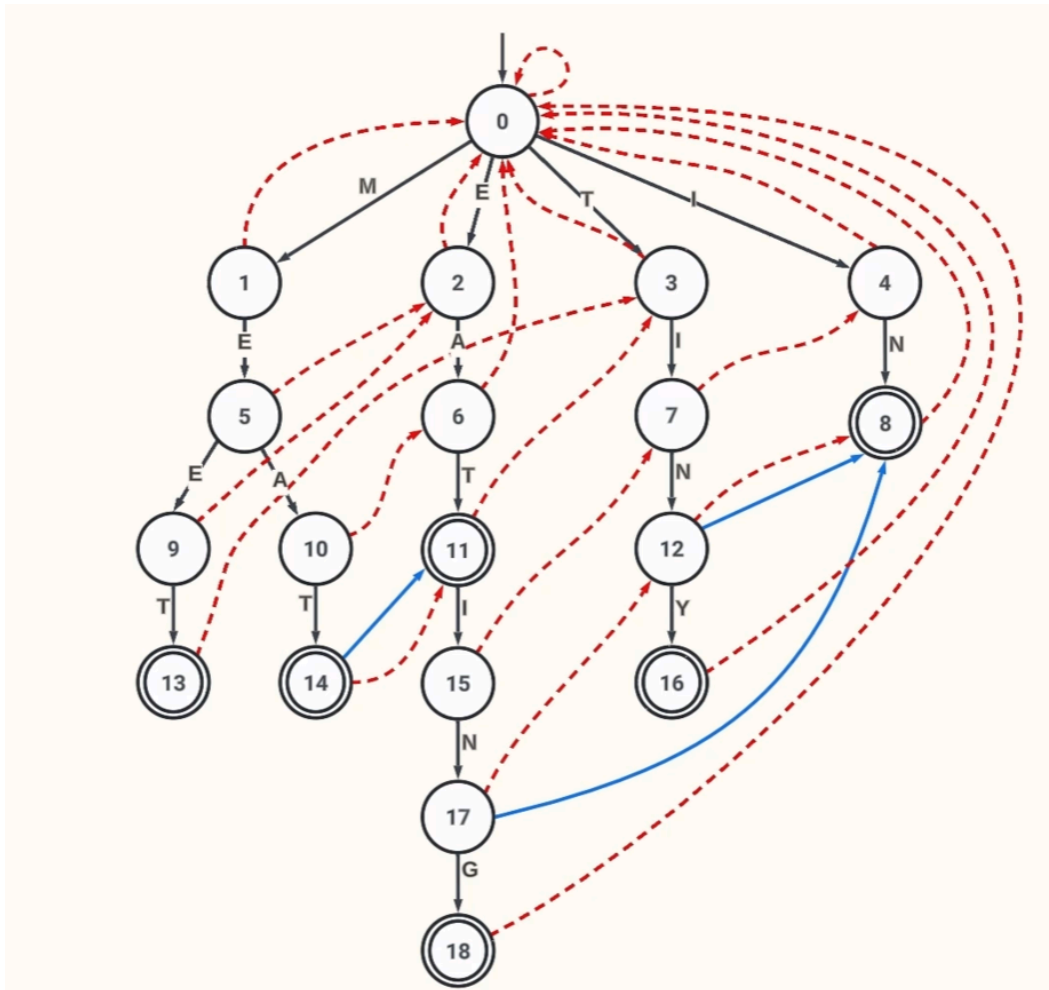
Gambar 2.1. *Levenshtein Distance*

(Sumber:

<https://medium.com/@ethannam/understanding-the-levenshtein-distance-equation-for-beginners-c4285a5604f0>)

2.5. Algoritma Aho-Corasick

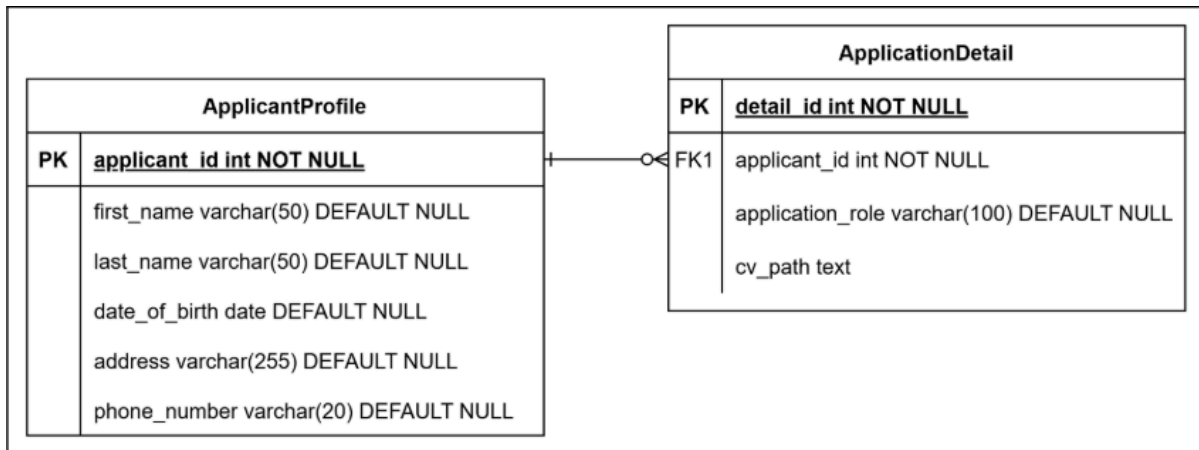
Algoritma Aho-Corasick, diciptakan oleh Alfred V. Aho dan Margaret J. Corasick, merupakan algoritma pencocokan string dengan kompleksitas waktu linear $O(n)$ yang memanfaatkan penggunaan struktur data Trie (Merupakan turunan dari konsep struktur data Tree) sehingga pencarian lebih efisien. Pendekatan ini memanfaatkan Suffix dari sebuah pattern, sama seperti KMP dan BM, tetapi tidak melibatkan *bruteforce* iterasi setiap karakter pattern dengan kata yang dicari seperti kedua metode tersebut. Berikut merupakan contoh struktur Trie:

Gambar 2.2. *Aho-Corasick Trie Example*

Sebuah node merepresentasikan sebuah karakter pada list of patterns. MEAT dan MEET memiliki dua karakter awal, sedangkan karakter ketiga pada dua pattern berbeda, sehingga Trie akan memecah menjadi dua branch berbeda. Proses ini dilakukan berulang kali hingga semua pattern telah dimasukkan kedalam Trie. Setelah itu, setiap node pada Trie akan dilakukan pencarian fail links berdasarkan proper suffix dari pattern. Caranya melakukan BFS dari root node untuk mencari fail link dari setiap node. Fail link direpresentasi sebagai warna merah, dimana node pada depth dibawah root node memiliki fail link yang merujuk pada root node. Node lain, seperti Node 10 merujuk pada Node 6, dimana Node 6 cocok dengan proper suffix yaitu EA dari MEAT. Panah berwarna biru menunjukkan output link yang menunjukkan bahwa hasil pencarian ditemukan keluaran. Sebelum pencarian dimulai, dilakukan preprocessing pembentukan Trie sehingga pada tahap pencarian sebenarnya akan melibatkan waktu linear dan tidak perlu membangun Trie lagi.

2.6. Penjelasan *Framework Aplikasi*

Aplikasi ini berbasis Python baik dari backend maupun tampilan antarmuka. Tampilan antarmuka dikembangkan menggunakan framework PyQt dan menggunakan toolkit PySide6. Selain itu, aplikasi memanfaatkan database MySQL, yang diakses dan diinisialisasi secara lokal dengan spesifikasi database sebagai berikut:



BAB III

ANALISIS PEMECAHAN MASALAH

3.1. Langkah-Langkah Pemecahan Masalah

Pencarian CV yang sesuai dengan pattern input dari user dimulai dengan membaca informasi dari database mengenai hubungan pelamar dan path ke CV File yang “dikumpul” oleh pelamar. Setelah database berhasil dimuat kepada aplikasi, user dapat menginput pattern yang ingin dicari terhadap keseluruhan database, input bisa berupa: “react, react native” yang berarti mencari CV dengan kata “react” atau dengan kata “react native”. User kemudian memilih algoritma pencocokan string dan menekan tombol Search untuk melakukan pencarian. Setelah pencarian selesai, user ditampilkan hasil dari pattern matching beserta applicant yang terlibat. User juga diberikan tombol summary yang memberikan ringkasan dari CV pelamar dengan kategori seperti “Experience” dan “Skill” yang memanfaatkan regex untuk mengekstrak data dari CV.

3.2. Proses Pemetaan Masalah

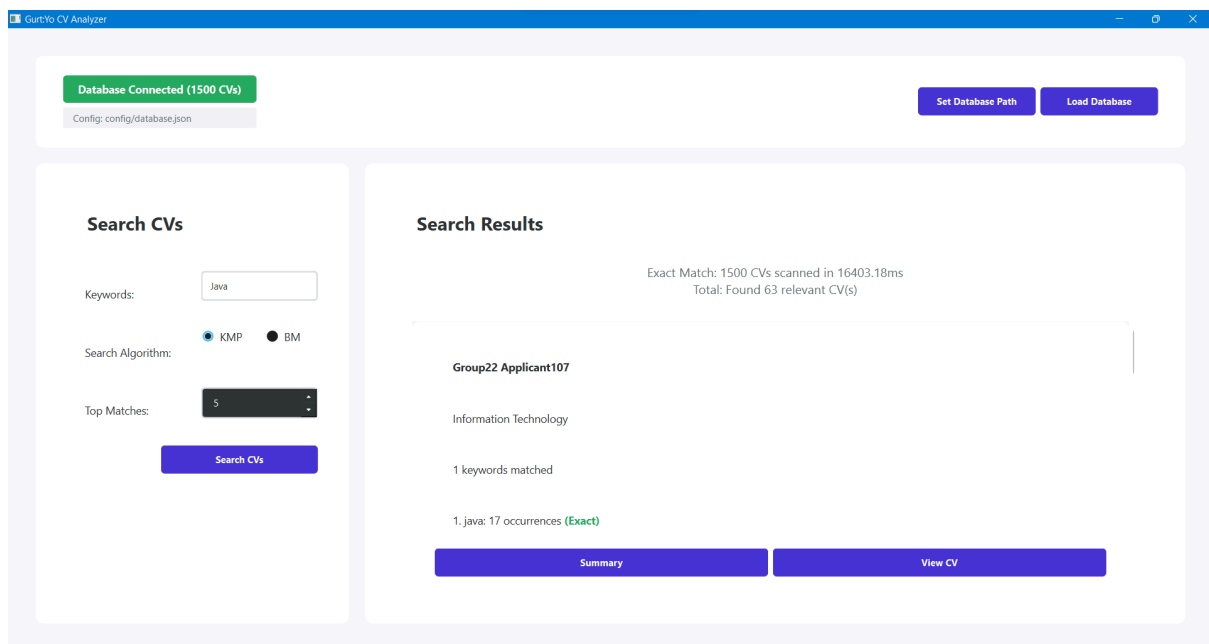
Database memiliki kolom yang mengarah ke file pdf untuk suatu lamaran. File tersebut dibaca oleh program. Teks dari file tersebut diekstrak lalu diolah dari teks mentah menjadi teks yang bersih yang bisa dibaca oleh program. Teks yang bersih akan digunakan untuk dilakukan searching oleh semua algoritma.

Ketika menggunakan algoritma KMP, LPS dihitung terlebih dahulu untuk membantu dalam pencarian. Ketika menggunakan BM, tabel *bad character* dihitung terlebih dahulu untuk membantu dalam pencarian. Ketika menggunakan AC, Node beserta *failure links* dibentuk terlebih dahulu untuk membantu dalam pencarian. Apabila ketiga algoritma tersebut gagal, algoritma fuzzy akan dipanggil dan algoritma tersebut langsung membaca teks tanpa membuat semacam *heuristik* pembantu terlebih dahulu.

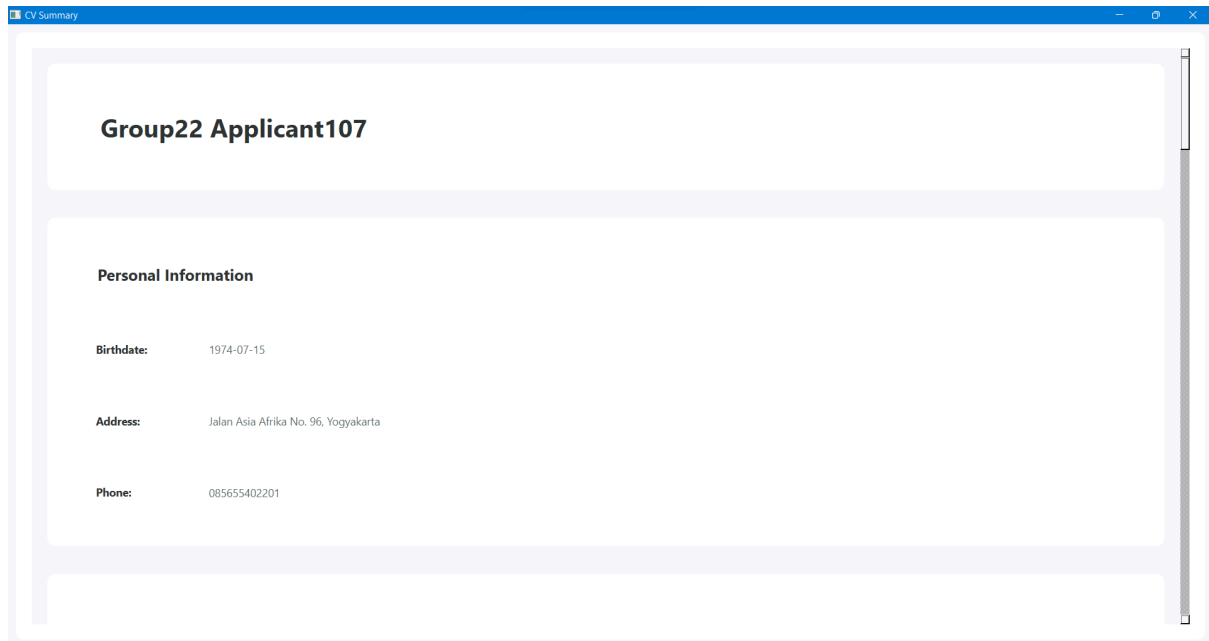
Regex juga membaca dari teks yang bersih dan melakukan search untuk mencari section-section. Section yang dicari sudah ditetapkan dalam sebuah variabel dan Regex memeriksa apakah kata ada dalam variabel tersebut atau tidak. Jika ada, ia akan memulai pengambilan kata-kata dan menyimpannya untuk diteruskan ke antarmuka.

3.3. Arsitektur Aplikasi

Aplikasi dibangun menggunakan arsitektur berlapis yang memisahkan logika aplikasi, logika pencarian, serta akses data dan membantu memudahkan pengembangan aplikasi. File [main.py](#) bertanggung jawab untuk interaksi pengguna dan menjadi antarmuka utama yang memberikan pilihan-pilihan untuk pemilihan database, pengaturan pencarian CV, serta hasil pencarian CV. File [SummaryWindow.py](#) menjadi antarmuka untuk menampilkan ringkasan dari CV yang dipilih. Penghubungan *backend* dengan antarmuka dilakukan oleh [main.py](#) sekaligus mengoordinasikan seluruh proses. Antarmuka dari aplikasi sebagai berikut



Gambar 2. Tampilan antarmuka utama aplikasi



Gambar 3. Tampilan ringkasan CV aplikasi

Pada backend pencarian kata kunci, implementasi fungsionalitas pencarian menggunakan *Strategy Design Pattern* yang memudahkan implementasi serta membuat implementasi menjadi lebih fleksibel. `SearchStrategy.py` mendefinisikan sebuah *interface* untuk mengimplementasikan metode *search* untuk masing-masing algoritma. `KMPStrategy.py`, `BMStrategy.py`, `ACStrategy.py`, serta `FuzzyStrategy.py` menjadi adapter untuk masing-masing kelas pencarian di `KMP.py`, `BM.py`, `AC.py`, `ACTrie.py` dan `Fuzzy.py`. Lalu `Search.py` bertindak sebagai “konteks” yang menggunakan algoritma yang dipilih untuk menjalankan pencarian.

Pada backend pengeklstrak informasi penting, program akan mengambil teks mentah dari file pdf melalui file `ExtractCV.py`. Setelah teks mentah berhasil didapatkan, teks akan dibaca menggunakan *Regular Expressions* (RegEx) untuk mengidentifikasi dan mengeklstrak bagian-bagian informasi yang terstruktur dari teks mentah tersebut. Teks yang sudah di-ekstrak kemudian akan ditampilkan pada `SummaryWindow`. Implementasi RegEx dilakukan melalui file `RegEx.py`.

Backend database diatur melalui `Database.py`. Komponen tersebut mengelola semua interaksi aplikasi dengan MySQL seperti membuat koneksi, mengambil daftar semua CV, hingga mengambil detil profil CV. Basis data menyimpan detil lamaran (id lamaran, id pelamar, jabatan yang ingin dilamar, dan letak file CV) dan detil pelamar (id pelamar, nama depan, nama belakang, tanggal lahir, alamat, dan nomor telepon).

3.4. Alur Kerja Aplikasi

Pertama-tama, pengguna menjalankan aplikasi utama. Pengguna akan menekan tombol *Load Database* untuk menyuruh aplikasi membuat koneksi dengan basis data. Pengguna kemudian memasukkan kata kunci yang dipisahkan dengan koma, memilih algoritma pencarian, dan memulai pencarian. Aplikasi akan melakukan pencarian dengan cara mendapatkan semua file CV dari basis data untuk mendapatkan daftar semua aplikasi yang tersimpan. Seluruh teks masing-masing file CV akan di-ekstrak dan setiap kata kunci akan dicari menggunakan algoritma yang dipilih dan algoritma fuzzy sebagai *fallback*. Setelah dicari, hasil akan ditampilkan dan diurutkan berdasarkan jumlah kemunculan. Dari sini pengguna bisa melihat ringkasan yang akan menampilkan data pelamar serta ringkasan CV yang berkaitan dan melihat CV secara langsung.

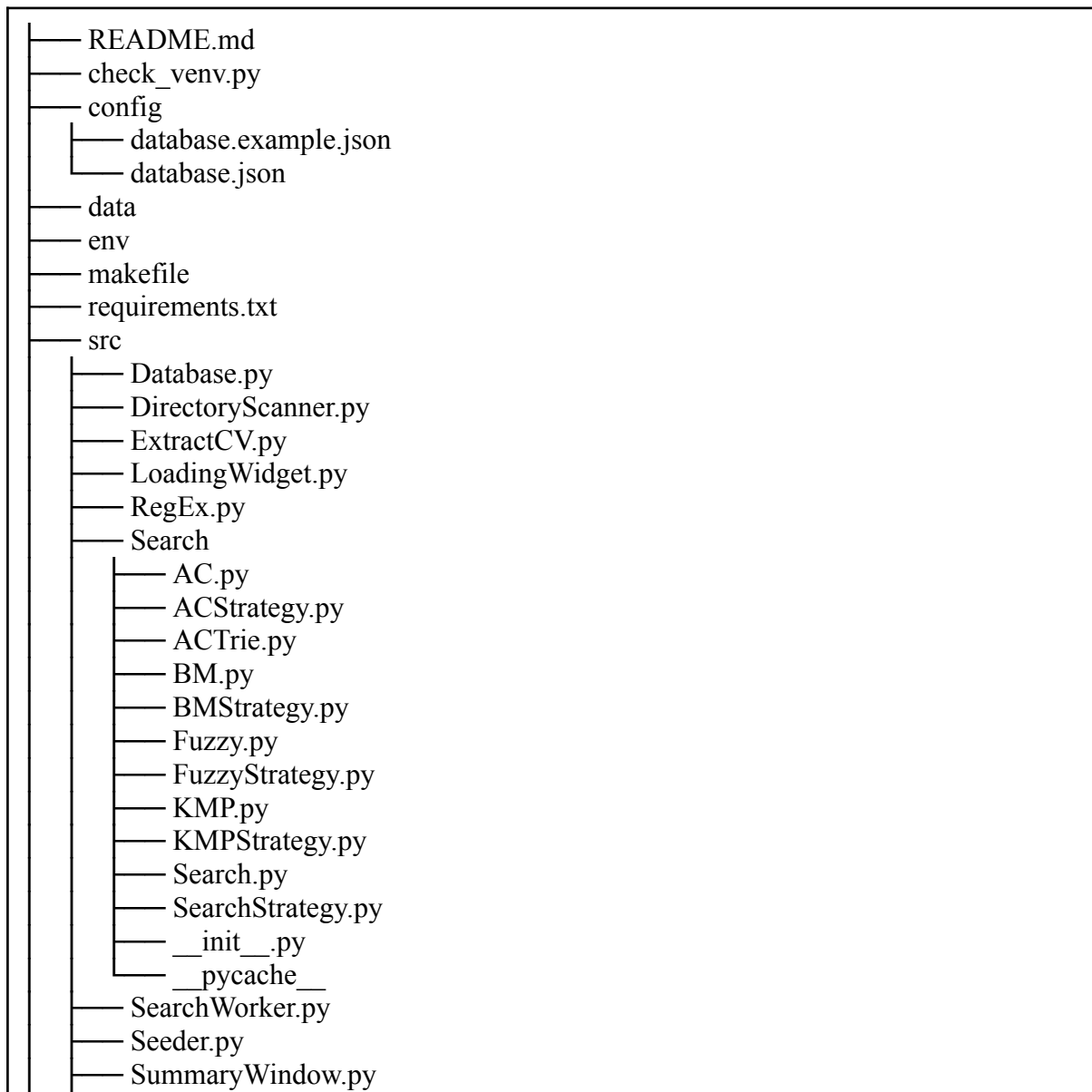
BAB IV

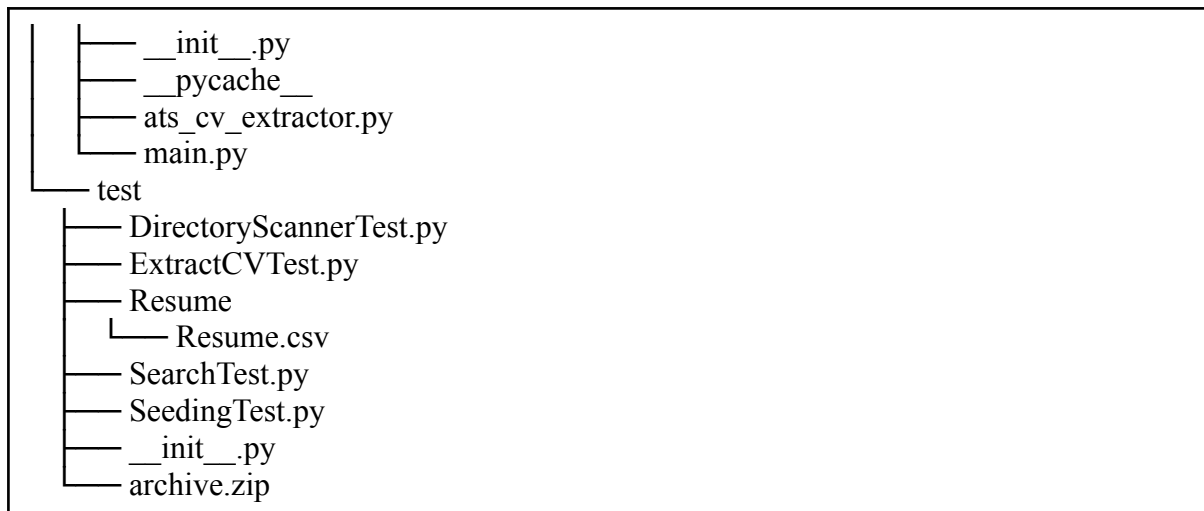
IMPLEMENTASI DAN PENGUJIAN

4.1. Spesifikasi Teknis Program

4.1.1. Struktur *Repository*

Berikut merupakan struktur *repository* program. Folder *src* berisi semua kode termasuk *main*, *Database*, algoritma *searching*, dan lainnya. Folder *config* berisi konfigurasi *database*. Folder *test* berisi file-file untuk *testing*. Folder *data* berisi semua file *CV* yang digunakan *database*. Pada *root folder* ada *makefile* untuk memudahkan dalam penjalanan program dan *testing-testing*.





4.1.2. Struktur Data

Database yang digunakan berbasis MySQL dengan bentuknya sebagai berikut:

```

MariaDB [tubes_3_stima]> show tables;
+-----+
| Tables_in_tubes_3_stima |
+-----+
| applicantprofile         |
| applicationdetail        |
+-----+
2 rows in set (0.010 sec)
  
```

Gambar 4. Tabel dalam basis data yang digunakan

```

MariaDB [tubes_3_stima]> describe applicantprofile;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| applicant_id   | int(11)       | NO   | PRI | NULL    | auto_increment |
| first_name     | varchar(50)   | YES  |     | NULL    |                |
| last_name      | varchar(50)   | YES  |     | NULL    |                |
| date_of_birth  | date          | YES  |     | NULL    |                |
| address        | varchar(255)  | YES  |     | NULL    |                |
| phone_number   | varchar(20)   | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.107 sec)
  
```

Gambar 5. *Fields* dalam tabel 'applicantprofile'

```

MariaDB [tubes_3_stima]> describe applicationdetail;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| detail_id      | int(11)       | NO   | PRI | NULL    | auto_increment |
| applicant_id   | int(11)       | NO   | MUL | NULL    |                |
| application_role | varchar(100)  | YES  |     | NULL    |                |
| cv_path        | text          | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.016 sec)

```

Gambar 6. *Fields* dalam tabel 'applicationdetail'

Mayoritas penyimpanan data dalam program ini menggunakan dictionary untuk memudahkan pencarian dan penyimpanan data. Selain itu, banyak variabel yang disimpan dalam bentuk string terutama dalam penyimpanan teks dan .

4.1.3. Fungsi dan Prosedur

Berikut merupakan penjelasan mengenai fungsi dan prosedur yang mengatur logic program

1. [main.py](#)

• Initializer

Initializer	Deskripsi
def __init__(self)	Fungsi yang mengatur tampilan antarmuka

• Metode

Metode	Deskripsi
def set_database_path(self)	Fungsi untuk menetapkan letak file konfigurasi database
def load_database(self)	Fungsi untuk menghubungkan dengan database
def perform_search(self)	Fungsi untuk memulai pencarian
def show_loading_widget(self)	Fungsi untuk menunjukkan program sedang memuat
def clear_results_area(self)	Fungsi yang menghapus kartu-kartu hasil pencarian
def cancel_search(self)	Fungsi untuk membatalkan pencarian

def on_search_progress(self, current, total, current_cv)	Dijalankan bersama dengan fungsi show_loading_widget, menjadi progress bar ketika program sedang memuat
def on_search_completed(self, results, timing_data, total_scanned, total_found) def on_search_finished(self)	Mengubah antarmuka dan mengeluarkan hasil pencarian, melakukan cleanup, dan memanggil fungsi pembuatan kartu CV
def on_search_error(self, error_message)	Fungsi yang dipanggil ketika muncul kesalahan dalam pencarian
def create_cv_card(self, detail_id, applicant_id, name, application_role, cv_path, matched_keywords_data)	Fungsi untuk membuat kartu masing-masing data pelamar
def show_summary(self, detail_id)	Fungsi untuk melihat ringkasan detail pelamar. Fungsi memanggil window lain
def view_cv(self, name, cv_path)	Fungsi untuk membuka file pdf pelamar

2. SummaryWindow.py

• Initializer

Initializer	Deskripsi
def __init__(self, details)	Fungsi yang mengatur tampilan antarmuka ringkasan serta menetapkan informasinya

• Metode

Metode	Deskripsi
def create_section(self, title, items)	Fungsi pembuat <i>section</i> untuk antarmuka (Summary, Skills, Experience, Education)

3. ExtractCV.py

• Atribut

Atribut	Deskripsi
variable raw_text	Variabel untuk menyimpan teks mentah
variable clean_text	Variabel untuk menyimpan teks yang sudah dibersihkan menjadi <i>continuous string</i>
variable pdf_path	Variabel untuk menyimpan letak file pdf

• Initializer

Initializer	Deskripsi
def __init__(self, pdf_path)	Menetapkan serta menginisialisasi atribut

- **Metode**

Metode	Deskripsi
def get_raw_text(self)	Mengembalikan teks mentah
def get_cleaned_text(self)	Mengembalikan teks bersih
def get_pdf_path(self)	Mengembalikan path lokasi file PDF
def set_pdf_path(self, new_path)	Menetapkan path lokasi file PDF baru
def extract_all_text(self)	Meng-ekstrak semua teks dari file PDF
def to_continuous_string(self)	Mengubah teks menjadi <i>lowercase</i> tetapi tetap menyimpan beberapa whitespace
def extract(self)	Konversi pdf menjadi <i>continuous string</i>

4. [RegEx.py](#)

- **Atribut**

Atribut	Deskripsi
variable section_patterns	Variabel untuk menyimpan jenis-jenis section
variable all_stop_patterns	Variabel untuk menyimpan nama-nama section sebagai penghalang

- **Metode**

Initializer	Deskripsi
def find_section_by_pattern(text, patterns)	Fungsi untuk mencari <i>section</i> pada teks dan mengembalikan teks serta posisinya
def extract_section_by_pattern(text, target_patterns, all_stop_patterns)	Mengekstrak teks untuk suatu <i>section</i> berdasarkan keyword teks
def extract_all_details(text)	Mengekstraksi semua <i>section</i> yang dicari serta header-header penghalang

5. Database.py

- **Initializer**

Initializer	Deskripsi
def __init__(self, config_path: str)	Menetapkan identitas untuk mengakses <i>database</i>

- **Metode**

Metode	Deskripsi
def set_path_config(self, config_path: str)	Menetapkan file konfigurasi <i>database</i>
def create_connection(self)	Mencoba membuat koneksi dengan <i>database</i>
def close_connection(self)	Menutup koneksi dengan <i>database</i>
def get_summary_details_by_id(self, detail_id)	Mengambil semua informasi <i>database</i> berdasarkan query
def get_all_cv_data(self)	Mengambil semua data lamaran dari <i>database</i>

6. Search.py

- **Atribut**

Atribut	Deskripsi
variable strategies	Variabel dictionary untuk menyimpan algoritma-algoritma pencarian
variable successStrategy	Variabel untuk menyimpan algoritma mana yang berhasil menemukan <i>keyword</i>

- **Initializer**

Initializer	Deskripsi
def __init__(self)	Menetapkan algoritma-algoritma pencarian dalam variable strategies dan menetapkan successStrategy sebagai None

- **Metode**

Metode	Deskripsi
def search(self, strategy_name, text, pattern)	Fungsi pencarian utama berdasarkan algoritma yang dipilih. Fungsi ini akan memanggil fungsi-fungsi Search lainnya pada BM.py, KMP.py, dan Fuzzy.py

7. BM.py

- **Atribut**

Atribut	Deskripsi
variable cv	Variabel sebagai instansi dari ExtractCV
variable pattern	Variabel untuk menyimpan keyword yang ingin dicari
variable bad_char	Variabel untuk menyimpan tabel heuristic dari pattern

- **Initializer**

Initializer	Deskripsi
def __init__(self, pattern, cv)	Menetapkan variabel serta menginisialisasi kelas BM

- **Metode**

Metode	Deskripsi
def set_pattern(self, pattern)	Menetapkan keyword
def get_pattern(self)	Mengembalikan keyword
def set_cv(self, cv)	Menetapkan instansi ExtractCV baru
def get_cv(self)	Mengembalikan instansi ExtractCV baru
def bad_char_heuristic(self, pattern)	Menghitung tabel heuristic dari pattern
def search(self)	Fungsi pencarian utama

8. KMP.py

- **Atribut**

Atribut	Deskripsi
variable cv	Variabel sebagai instansi dari ExtractCV
variable pattern	Variabel untuk menyimpan keyword yang ingin dicari
variable lps	Variabel untuk menyimpan array longest prefix suffix (LPS)

- **Initializer**

Initializer	Deskripsi
def __init__(self, pattern, cv)	Menetapkan variabel serta

	menginisialisasi kelas KMP
--	----------------------------

- **Metode**

Metode	Deskripsi
def set_pattern(self, pattern)	Menetapkan keyword
def get_pattern(self)	Mengembalikan keyword
def set_cv(self, cv)	Menetapkan instansi ExtractCV baru
def get_cv(self)	Mengembalikan instansi ExtractCV baru
def compute_lps(self, pattern)	Menghitung array LPS
def search(self)	Fungsi pencarian utama

9. Fuzzy.py

- **Atribut**

Atribut	Deskripsi
variable cv	Variabel sebagai instansi dari ExtractCV
variable pattern	Variabel untuk menyimpan keyword yang ingin dicari

- **Initializer**

Initializer	Deskripsi
def __init__(self, pattern, cv)	Menetapkan variabel serta menginisialisasi kelas Fuzzy

- **Metode**

Metode	Deskripsi
def set_pattern(self, pattern)	Menetapkan keyword
def get_pattern(self)	Mengembalikan keyword
def set_cv(self, cv)	Menetapkan instansi ExtractCV baru
def get_cv(self)	Mengembalikan instansi ExtractCV baru
def levenshtein_distance(self, s1, s2)	Menghitung jarak <i>Levenshtein</i> antara 2 string
def calculate_threshold_from_length(self, length)	Menghitung <i>threshold</i> berdasarkan panjang keyword

def search(self)	Fungsi pencarian utama
------------------	------------------------

10. ACTrie.py

• **Atribut**

Atribut	Deskripsi
variable children	Variabel untuk menyimpan children dari Node ACTrie
variable output	Variabel untuk menyimpan keyword
variable fail_link	Pointer ke link node yang gagal
variable root	Menyimpan ACTrieNode untuk ACTrie

• **Initializer**

Initializer	Deskripsi
ACTrieNode::def __init__(self)	Menetapkan variabel children, output, dan fail_link
ACTrieNode::def __init__(self)	Menetapkan variabel untuk ACTrieNode

• **Metode**

Metode	Deskripsi
def insert(self, pattern)	Menambah keyword ke node
def build_failure_links(self)	Membangun <i>failure links</i> pada node

11. AC.py

• **Atribut**

Atribut	Deskripsi
variable cv	Variabel untuk menyimpan instansi ExtractCV
variable ac_trie	Variabel untuk menyimpan instansi ACTrie

• **Initializer**

Initializer	Deskripsi
def __init__(self, pattern, cv)	Menginisialisasi kelas AC berdasarkan keyword dan instansi CV dan menetapkan variabel-variabel

- **Metode**

Metode	Deskripsi
def insert_pattern(self, pattern)	Menambah keyword ke node
def build_failure_links(self)	Membangun <i>failure links</i> pada node
def set_cv(self, cv)	Menetapkan instansi ExtractCV baru
def get_cv(self)	Mengembalikan instansi ExtractCV baru
def search(self)	Fungsi pencarian utama

12. SearchWorker.py

- **Atribut**

Atribut	Deskripsi
variable db	Variabel dictionary untuk menyimpan algoritma-algoritma pencarian
variable search_engine	Variabel untuk menyimpan algoritma mana yang berhasil menemukan <i>keyword</i>
variable keywords	Variabel untuk menyimpan semua keyword yang ingin dicari
variable algorithm	Variabel untuk menyimpan pilihan algoritma
variable top_n	Variabel untuk menyimpan jumlah kartu yang ditampilkan
variable _is_cancelled	Variabel untuk menyimpan apakah proses dibatalkan atau tidak

- **Initializer**

Initializer	Deskripsi
def __init__(self)	Menetapkan dan menginisialisasi atribut-atribut sebagai worker untuk multi-threading. Variabel <i>_is_cancelled</i> ditetapkan sebagai None.

- **Metode**

Metode	Deskripsi
def cancel(self)	Membatalkan fungsi pencarian

def run(self)	Memulai multithreading untuk pencarian. Thread pencarian utama terletak di tempat lain.
---------------	---

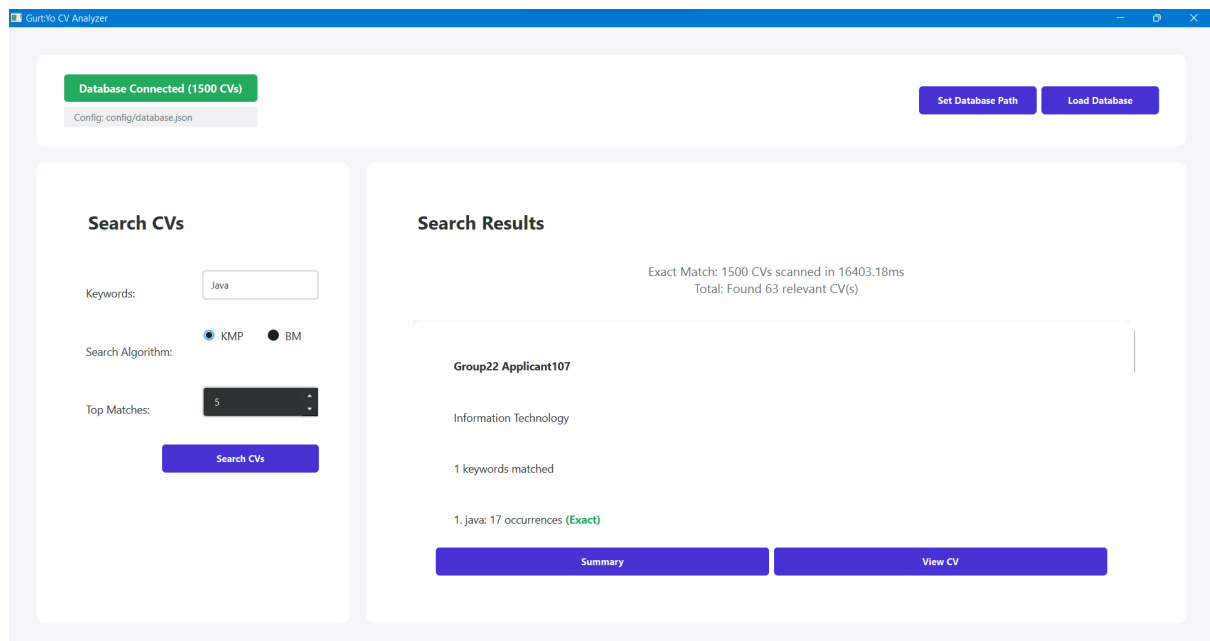
4.2. Cara Menggunakan Program

Pengguna pertama-tama memuat konfigurasi database yang ingin digunakan. Setelah itu, pengguna memasukkan *keyword* yang ingin dicari, memilih algoritma yang ingin digunakan, serta memasukkan jumlah kartu yang ingin ditampilkan. Pengguna kemudian dapat langsung melakukan pencarian. Setelah beberapa waktu, program akan menampilkan hasil pencariannya beserta jumlah file yang di-scan dan waktu yang dibutuhkannya. Pengguna kemudian dapat memilih untuk membuka ringkasan CV atau membuka CV secara langsung.

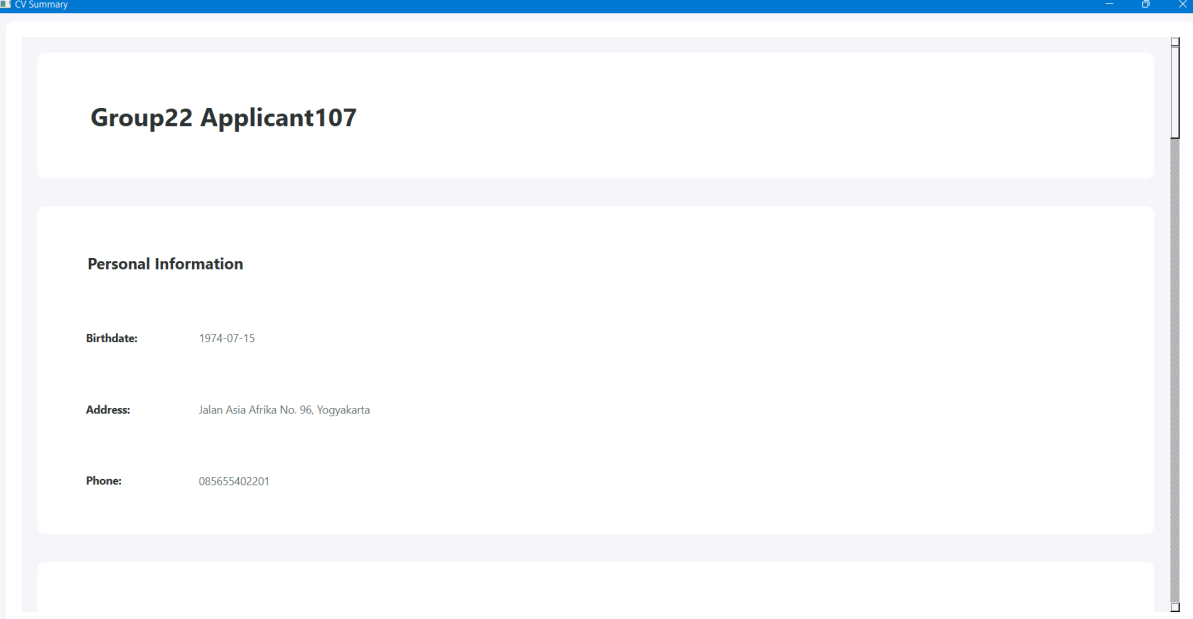
4.3. Hasil Pengujian

Berikut hasil pengujian pada program menggunakan data placeholder dari kami sendiri.

1. 1 Keyword, KMP



Gambar 4. Hasil pengujian 1 keyword menggunakan Metode KMP



Group22 Applicant107

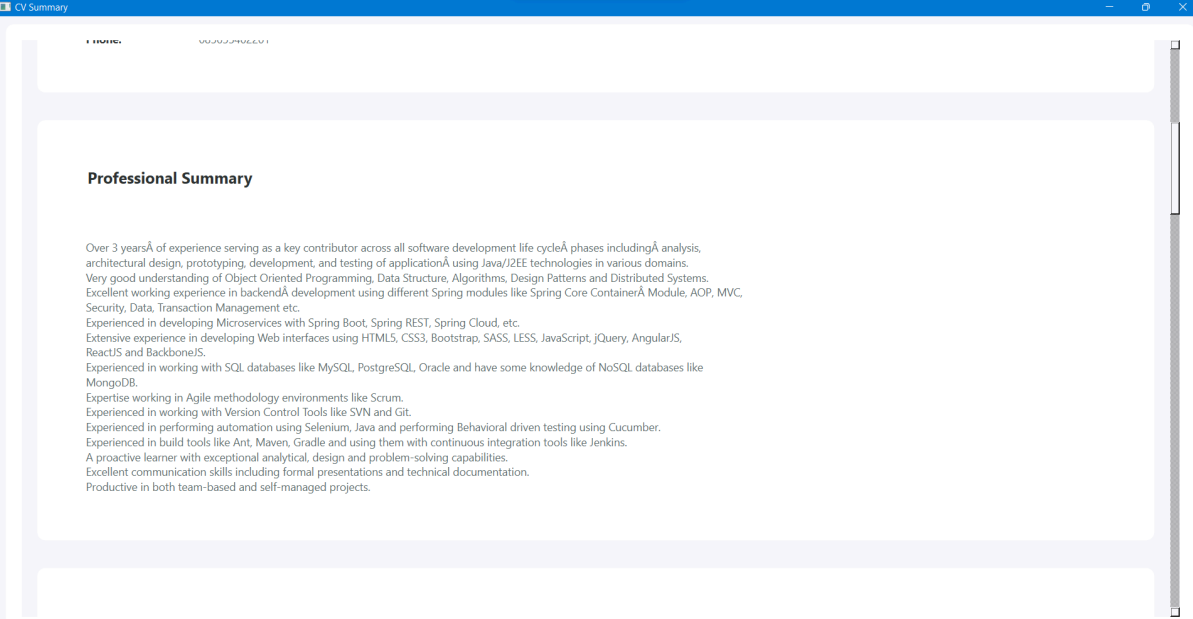
Personal Information

Birthdate: 1974-07-15

Address: Jalan Asia Afrika No. 96, Yogyakarta

Phone: 085655402201

Gambar 5. Ringkasan Data CV 1



Professional Summary

Over 3 years of experience serving as a key contributor across all software development life cycle phases including analysis, architectural design, prototyping, development, and testing of applications using Java/J2EE technologies in various domains. Very good understanding of Object Oriented Programming, Data Structure, Algorithms, Design Patterns and Distributed Systems. Excellent working experience in backend development using different Spring modules like Spring Core Container Module, AOP, MVC, Security, Data, Transaction Management etc.

Experienced in developing Microservices with Spring Boot, Spring REST, Spring Cloud, etc.

Extensive experience in developing Web interfaces using HTML5, CSS3, Bootstrap, SASS, LESS, JavaScript, jQuery, AngularJS, ReactJS and BackboneJS.

Experienced in working with SQL databases like MySQL, PostgreSQL, Oracle and have some knowledge of NoSQL databases like MongoDB.

Expertise working in Agile methodology environments like Scrum.

Experienced in working with Version Control Tools like SVN and Git.

Experienced in performing automation using Selenium, Java and performing Behavioral driven testing using Cucumber.

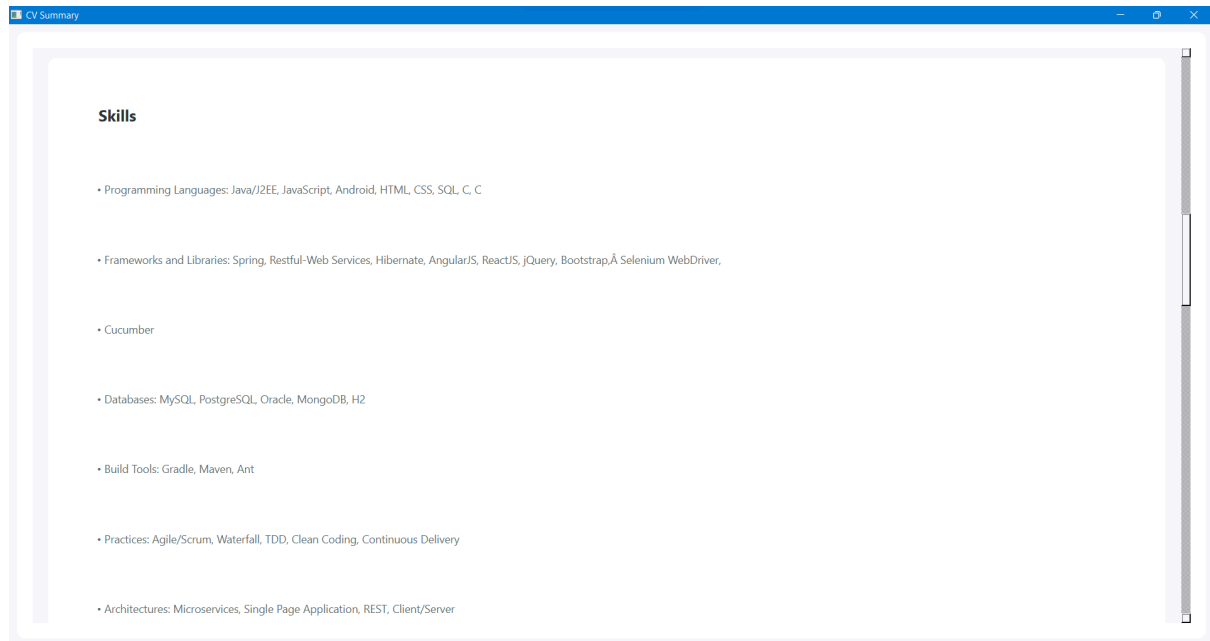
Experienced in build tools like Ant, Maven, Gradle and using them with continuous integration tools like Jenkins.

A proactive learner with exceptional analytical, design and problem-solving capabilities.

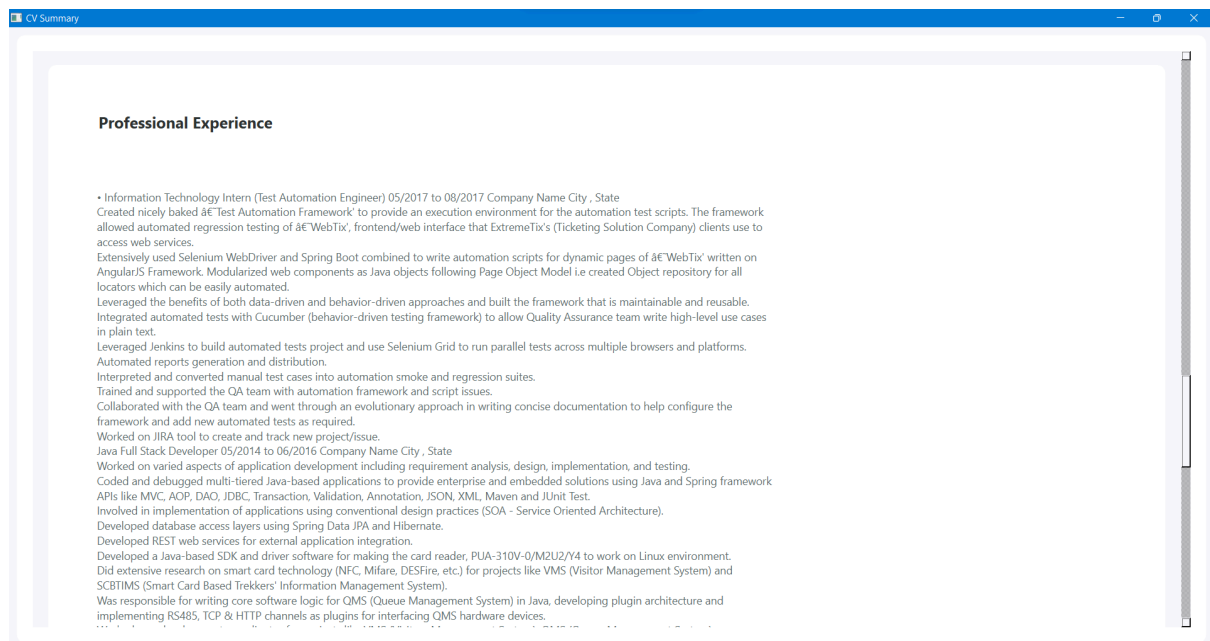
Excellent communication skills including formal presentations and technical documentation.

Productive in both team-based and self-managed projects.

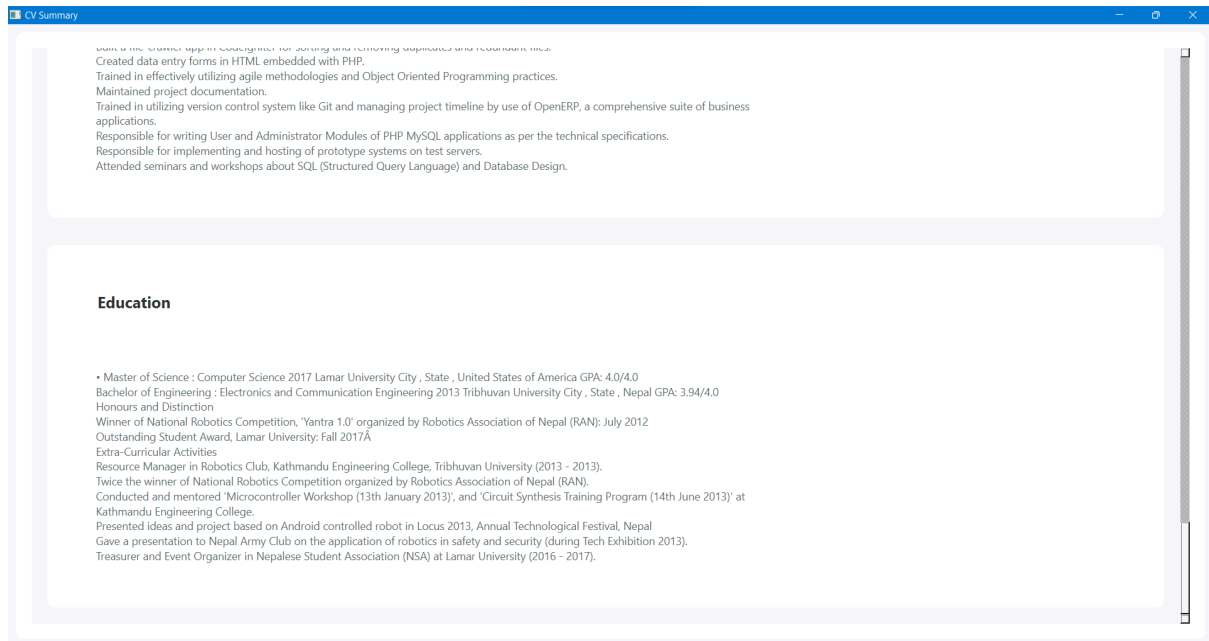
Gambar 6. Ringkasan Data CV 2



Gambar 7. Ringkasan Data CV 3

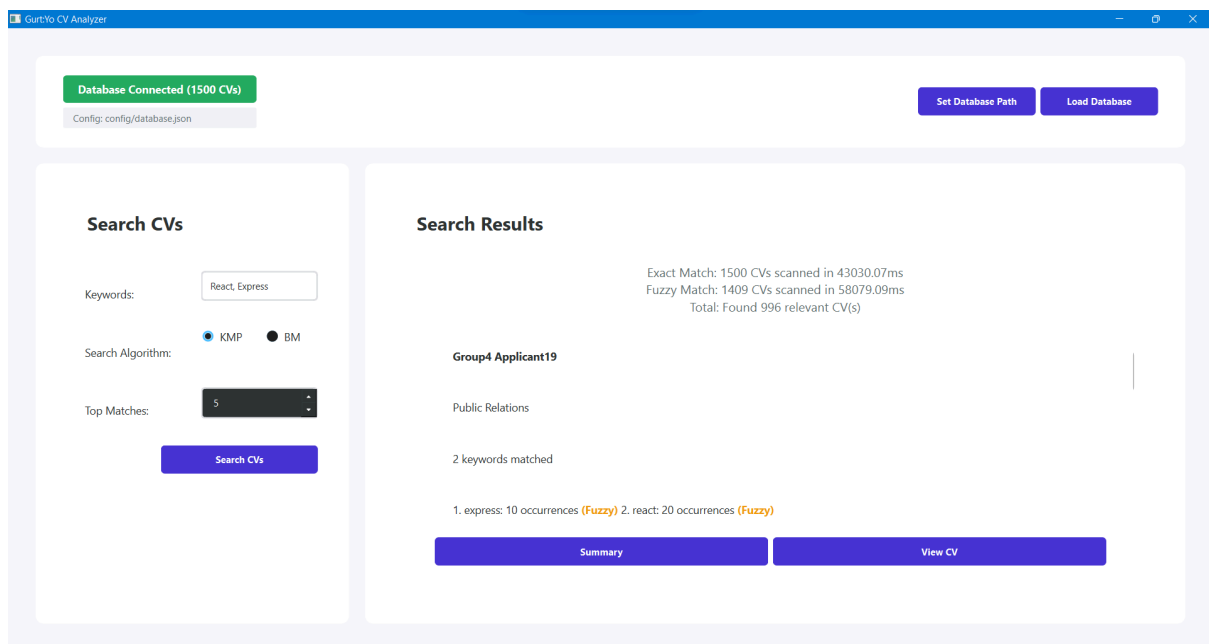


Gambar 8. Ringkasan Data CV 4



Gambar 9. Ringkasan Data CV 5

2. 2 Keyword, KMP



Gambar 10. Hasil pengujian 2 keyword menggunakan Metode KMP

Group4 Applicant19

Personal Information

Birthdate: 1998-09-23

Address: Jalan Pahlawan No. 88, Medan

Phone: 085619567737

Gambar 11. Ringkasan Data CV 1

Professional Summary

Objective: Seeking a professional bartender position with your company, which will require me to utilize my skills, abilities and experience to contribute and assist in ensuring the company's success. Summary: Experienced TABC and Food Handler Certified Special Event Bartender/Cocktail Server of 7 years. Skill and Knowledge:

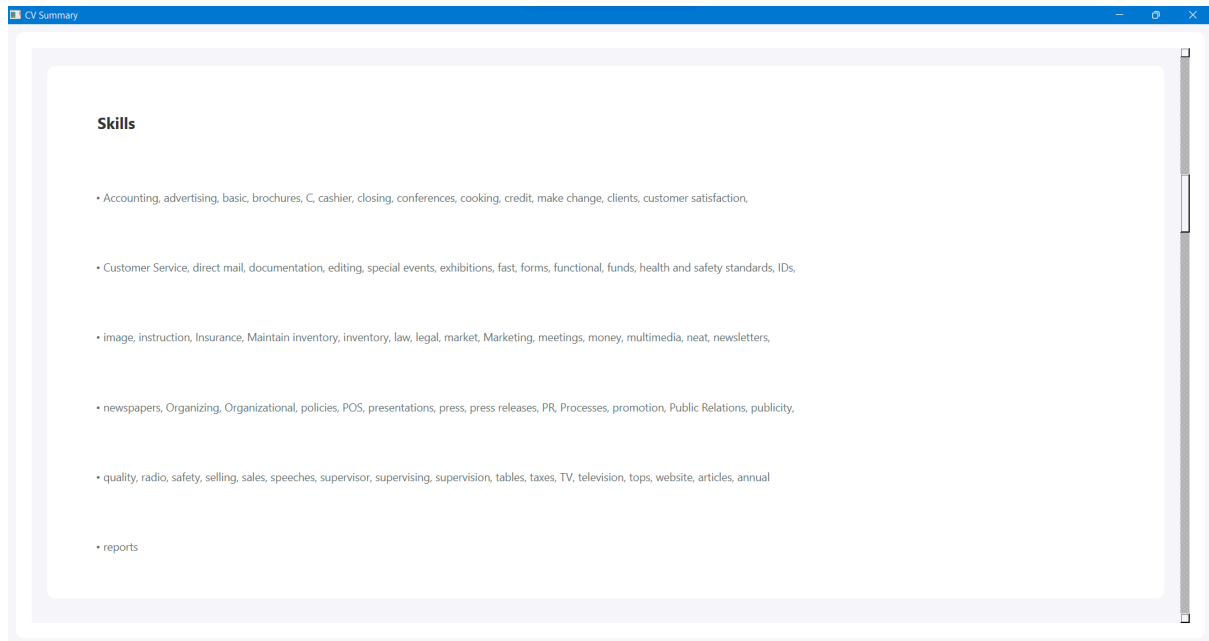
- Knowledge of principles and processes for providing outstanding customer and personal services
- Ability to multi-task
- Knowledge of food and beverage preparations, service standards, guest relations and etiquette
- Knowledge of the appropriate table settings and service ware
- Ability to stand for extended periods of time during a shift
- Ability to work shifts at off premise locations
- Possesses strong verbal and written communication skills
- Knowledge of beverage preparation and service of various alcoholic beverages

Personal Characteristics:

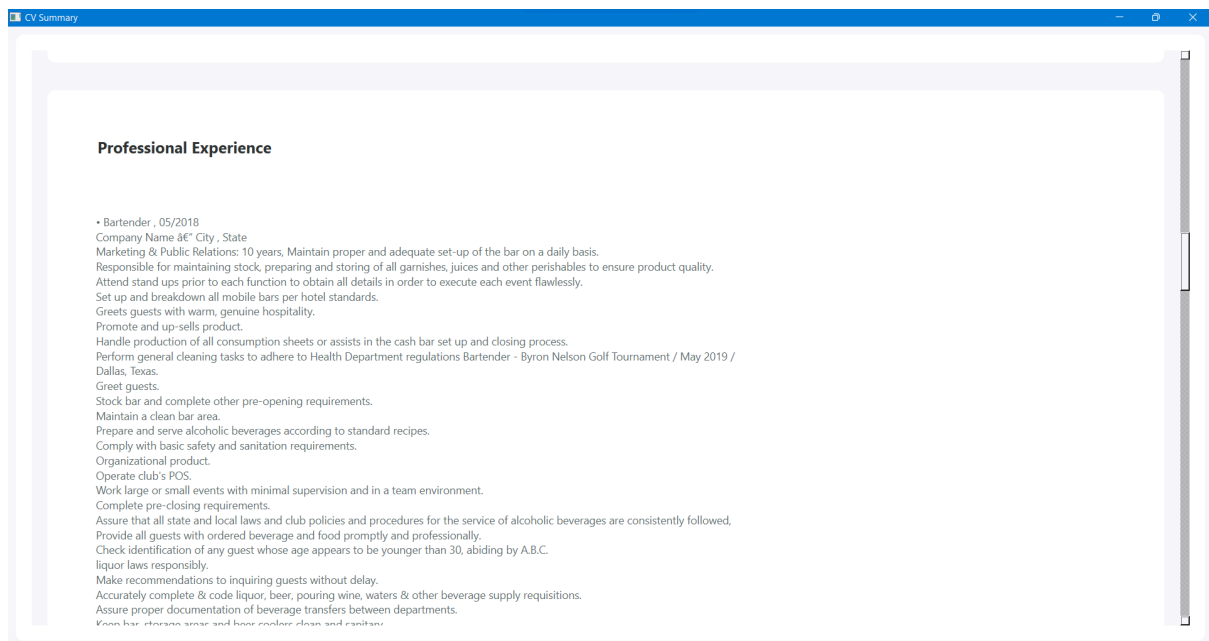
- Professional demeanor in appearance
- Confident manner of speaking
- Willingness to engage with guests
- Mental flexibility to perform different roles day to day
- Sense of urgency to complete tasks and service guests in a timely manner
- Ability to work with a sense of urgency on behalf of the membership without disrupting the atmosphere of the dining room
- Effortlessly polite, patient and courteous at all times
- Interacts well and effectively communicate with members, guests and staff
- Maintains neat and professional appearance
- Ability to follow directions from a supervisor
- Ability to work fast and safely
- Possesses good coordination and balance
- Personable, responsible and willing to work in a team environment
- Ability to properly execute the POS system

Capable bartender that is passionate about serving in warm, efficient and courteous manner while exceeding customer expectations and maintaining poise and sense of humor in demanding environments. Looking to bring history of exceptional service to growing restaurant. Professional Bartender with extensive knowledge of spirits and intuitive ability to engage customers of all types. Friendly, open and committed to building customer base by providing positive service experience.

Gambar 12. Ringkasan Data CV 2

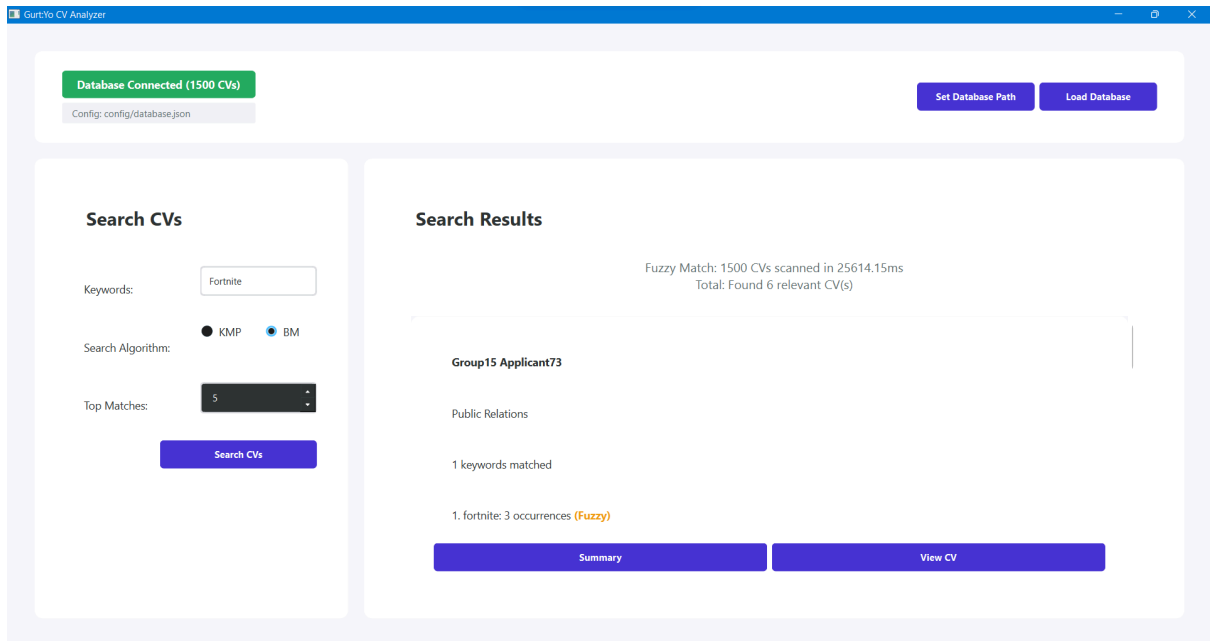


Gambar 13. Ringkasan Data CV 3

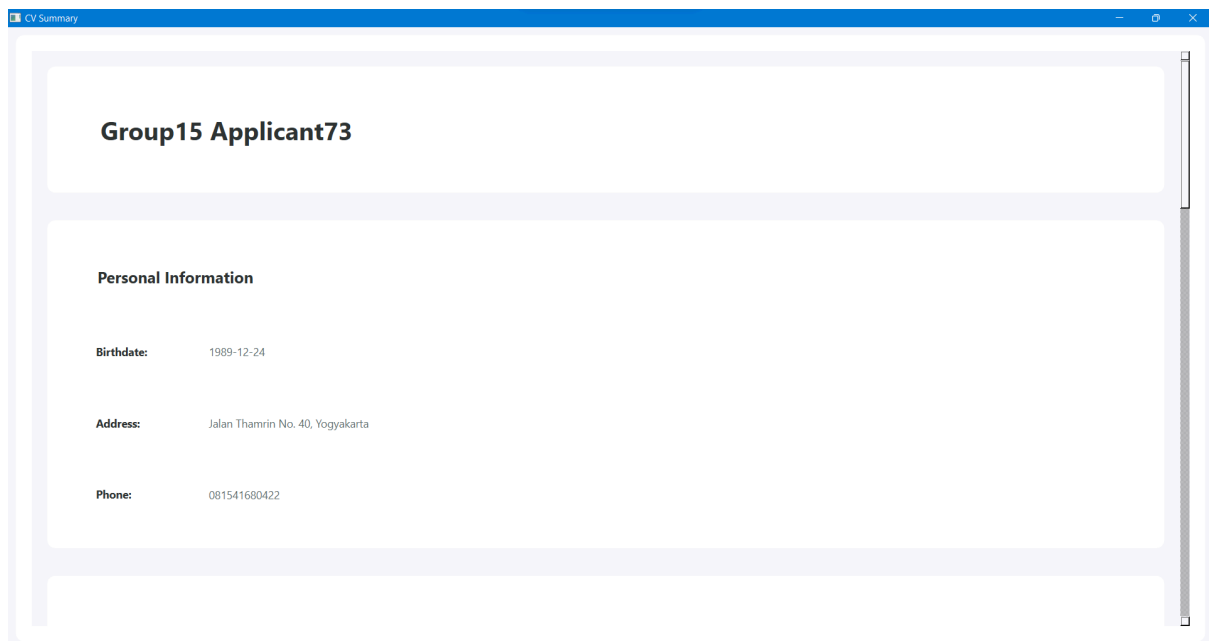


Gambar 14. Ringkasan Data CV 4

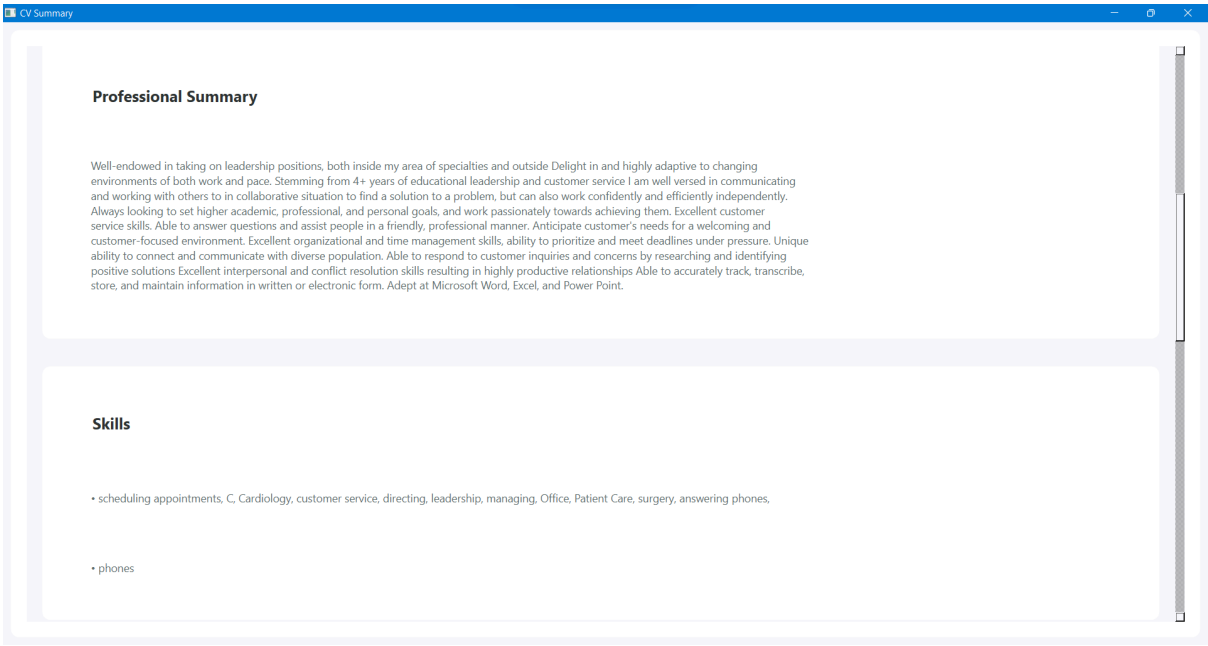
3. 1 Keyword, BM



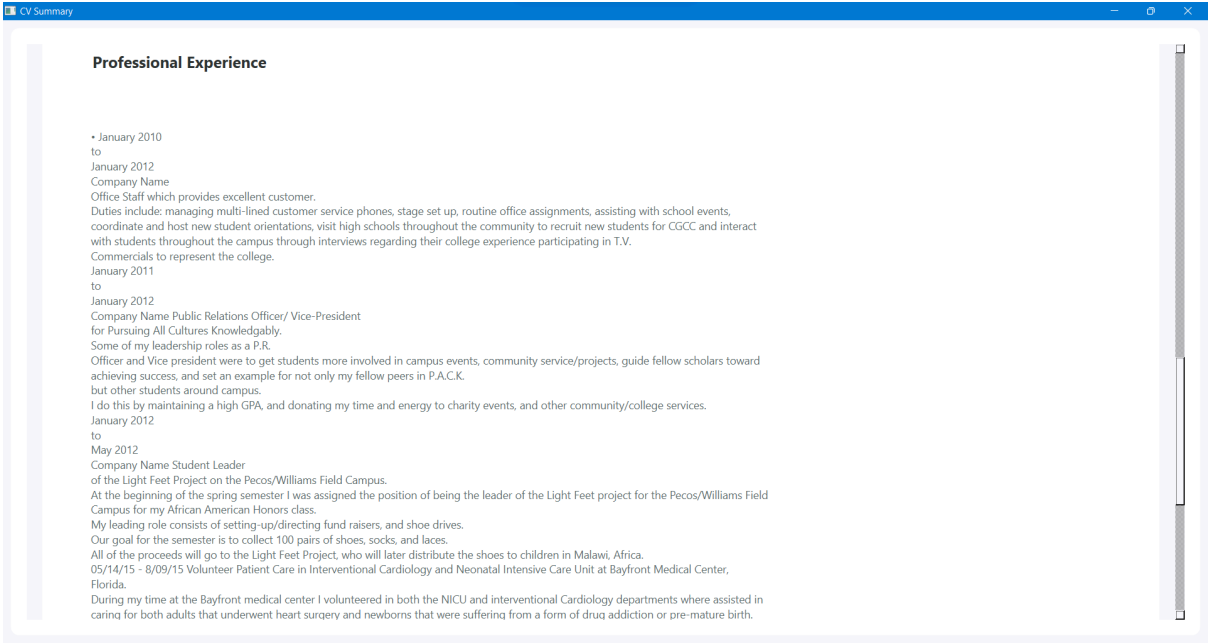
Gambar 15. Hasil pengujian 1 keyword menggunakan Metode BM



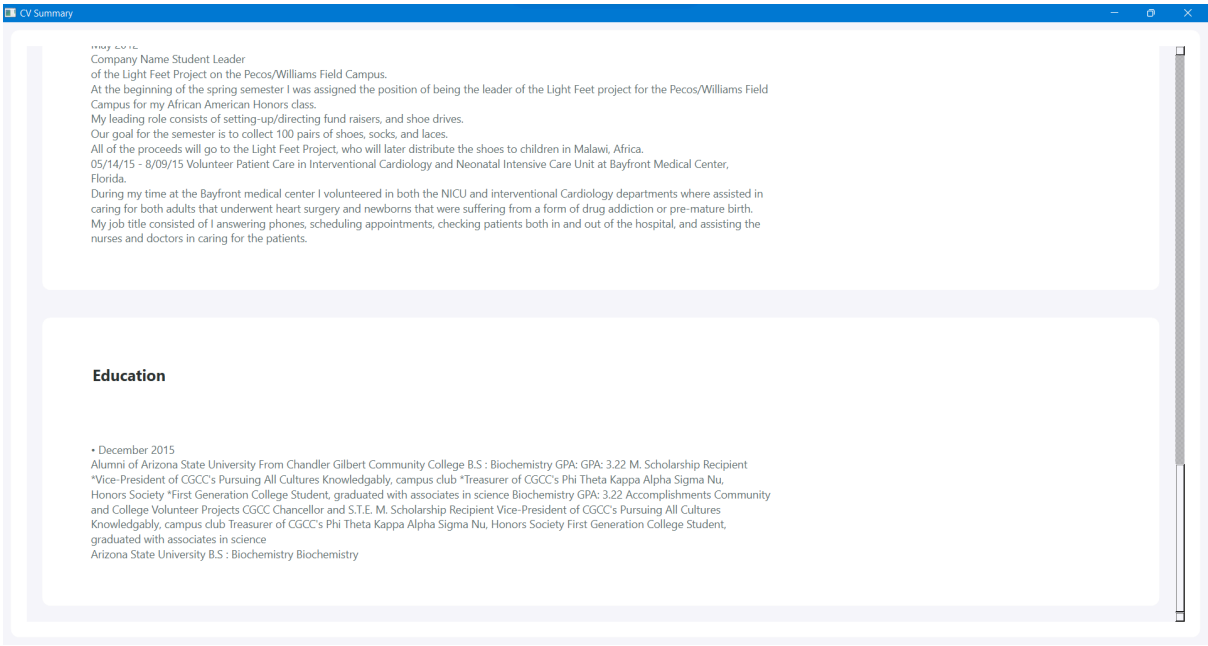
Gambar 16. Ringkasan Data CV 1



Gambar 17. Ringkasan Data CV 2

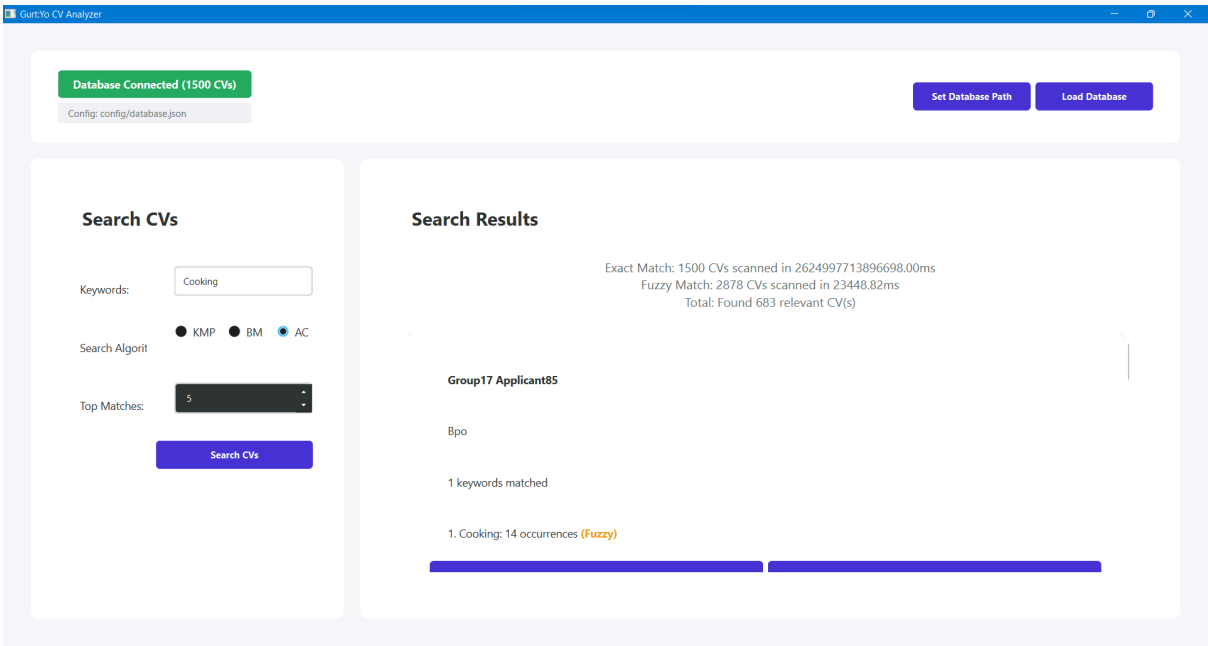


Gambar 18. Ringkasan Data CV 3



Gambar 19. Ringkasan Data CV 4

4. 1 Keyword, AC



Gambar 20. Hasil pengujian 1 keyword menggunakan Metode AC

Group17 Applicant85

Personal Information

Birthdate: 1994-05-17

Address: Jalan Gatot Subroto No. 124, Semarang

Phone: 081297489223

Gambar 21. Ringkasan Data CV 1

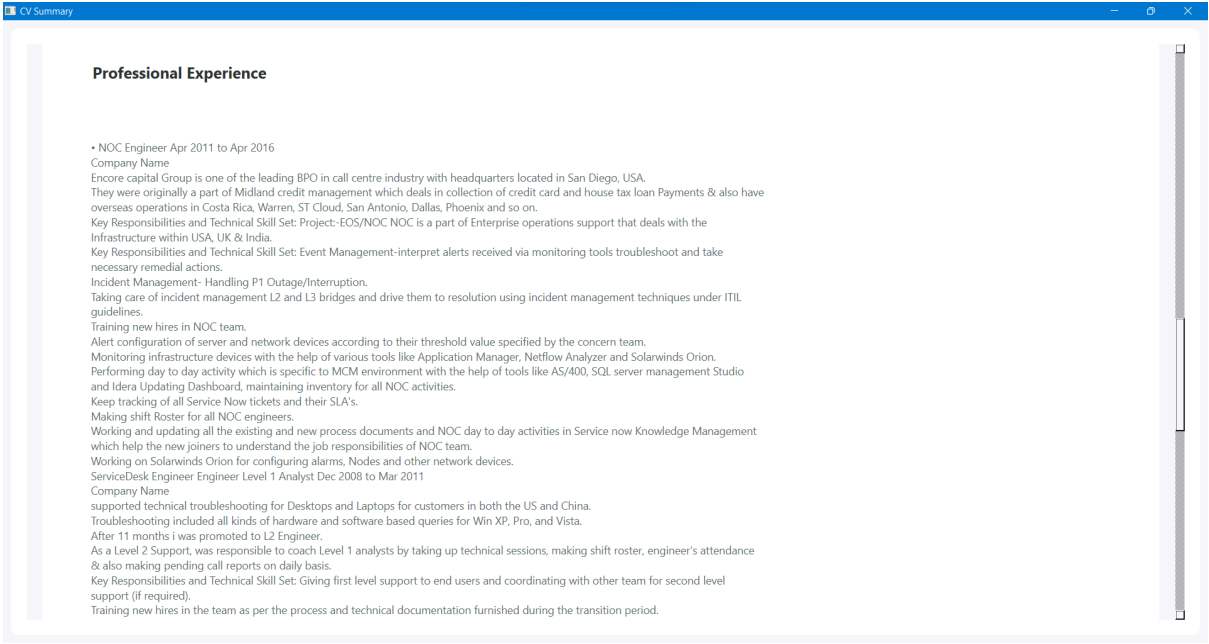
Professional Summary

To work for professional organization that gives an opportunity for growth and to contribute value based management talent along with the seeking guidance from proficient seniors to help me sharpen my skills. To be a part of the team that works in dynamically challenging environment for growth of the organization and helping in achieving goal congruency. Working as a NOC (Network Operation Center) Engineer in Encore Capital Group. That is a leading BPO in call center technology. Having Eight plus years of professional experience, I bring forth a unique set of knowledge and skills which include an in-depth understanding of Business Processes and Structured Methodologies, experience in the Customer Service industry, excellent understanding of Offshore Client requirements, professional education and valuable experience gained mostly while working in last couple of organisation.

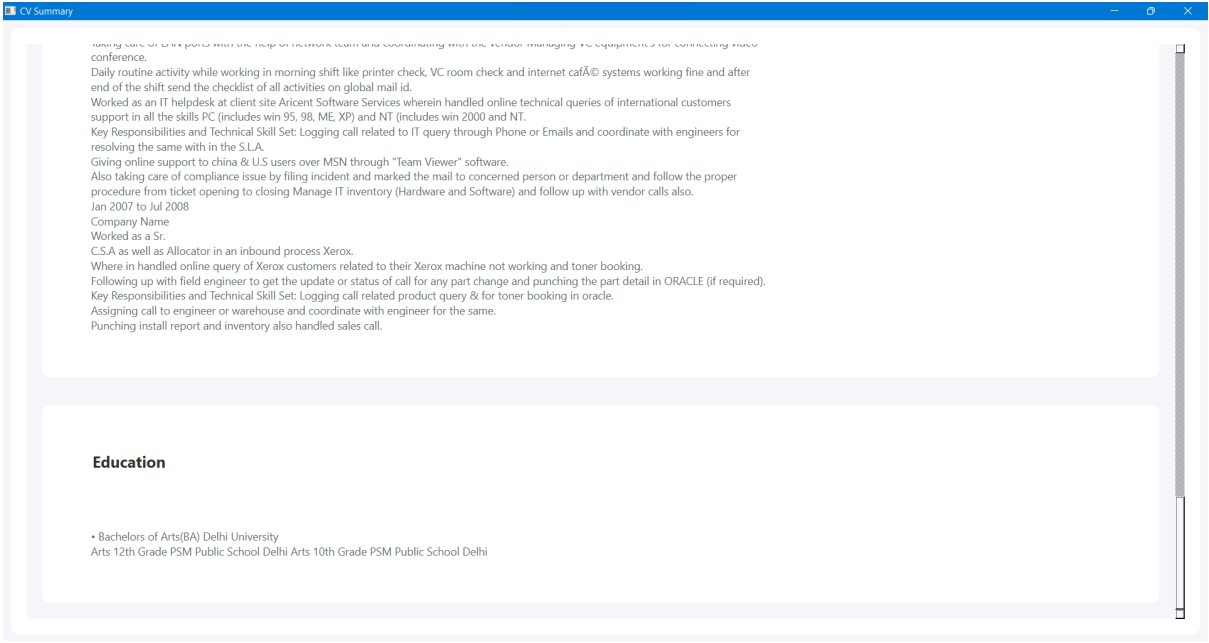
Skills

- a Level 2, analyst, Arts, AS/400, bridges, C, call centre, CCNA, Cisco, closing, coach, Computer Hardware, Hardware, credit, Client,
- Desktops, Documentation, Engineer, English, Event Management, filing, GUI, Hindi, ids, maintaining inventory, inventory, ITIL, ITIL v,
- Knowledge Management, LAN, Laptops, LEADERSHIP, Logging, Managing, access, Microsoft exchange, mail, MS Office, Office, Windows 7,

Gambar 22. Ringkasan Data CV 2



Gambar 23. Ringkasan Data CV 3



Gambar 24. Ringkasan Data CV 4

BAB V

KESIMPULAN DAN SARAN

5.1. Kesimpulan

Pattern matching adalah persoalan dengan aplikasi nyata yang luas, salah satunya dalam penyaringan informasi *Curriculum Vitae* (CV) pada *Applicant Tracking System* (ATS). Terdapat dua pendekatan dalam proses *pattern matching*, yakni eksak dan kabur (*fuzzy*). Algoritma Knuth-Morris-Pratt dan Boyer-Moore merupakan metode pencocokan string yang lebih efisien daripada metode naif dengan cara menghindari operasi perbandingan yang redundan, mengurangi jumlah operasi total. Pendekatan *fuzzy* melakukan perbandingan string dengan sebuah metrik yakni *Levenshtein Distance* untuk mengukur kemiripan dua string, memungkinkan pencocokan yang tidak eksak. Proses *pattern matching* dapat dioptimisasi menggunakan algoritma Aho-Corasick yang memanfaatkan struktur data trie untuk mengkonstruksi *finite-state machine*, memungkinkan pencarian dalam waktu linear.

Penerapan *pattern matching* direalisasikan dengan pembuatan aplikasi penyaring informasi CV untuk ATS. Aplikasi dikembangkan dalam bahasa Python dengan PyQT 6 dan *toolkit* PySide6 untuk antarmuka dan SQL untuk basis data. Aplikasi dapat menerima input kata kunci yang ingin dicari dalam kumpulan berkas CV berformat PDF lalu mengembalikan ringkasan informasi CV yang mengandung kata kunci tersebut beserta keterangan waktu eksekusi pencarian dan jumlah kemunculan kata kunci dalam CV.

5.2. Saran

Bagi pengembang berikutnya, implementasi aplikasi penyaring informasi CV ATS dapat dikembangkan dengan menambah fitur pada antarmuka yang mempermudah pengalaman pengguna, seperti fitur filter, pengurutan, atau rekomendasi pelamar berdasarkan spesifikasi kebutuhan pemberi kerja. Basis data juga dapat dikembangkan dengan melakukan *deployment* pada *cloud server* untuk sentralisasi akses data.

5.3. Refleksi

Tugas besar ini telah memberikan kami pengalaman yang sangat menekankan. Beban pengerjaan kami cukup stres apalagi ketika mengerjakannya di masa UAS. Kami menghadapi berbagai macam kendala seperti manajemen waktu, prioritas. Untuk kedepannya, diharapkan tugas besar ini bisa menjadi motivasi untuk kami agar kami lebih berusaha baik dalam pengerjaan tugas-tugas berikutnya.

LAMPIRAN

Link repository GitHub: github.com/19623248Git/Tubes3_gurt-yo

No	Poin	Ya	Tidak
1	Aplikasi dapat dijalankan.	✓	
2	Aplikasi menggunakan basis data berbasis SQL dan berjalan dengan lancar.	✓	
3	Aplikasi dapat mengekstrak informasi penting menggunakan Regular Expression (Regex).	✓	
4	Algoritma <i>Knuth-Morris-Pratt (KMP)</i> dan <i>Boyer-Moore (BM)</i> dapat menemukan kata kunci dengan benar.	✓	
5	Algoritma <i>Levenshtein Distance</i> dapat mengukur kemiripan kata kunci dengan benar.	✓	
6	Aplikasi dapat menampilkan <i>summary CV applicant</i> .	✓	
7	Aplikasi dapat menampilkan <i>CV applicant</i> secara keseluruhan.	✓	
8	Membuat laporan sesuai dengan spesifikasi.	✓	
9	Membuat bonus enkripsi data profil <i>applicant</i> .		✓
10	Membuat bonus algoritma <i>Aho-Corasick</i> .	✓	
11	Membuat bonus video dan diunggah pada Youtube.		✓

DAFTAR PUSTAKA

- Munir, R. (2025). Pencocokan string (String/pattern matching) [Course slides]. Program Studi Teknik Informatika STEI-ITB. [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/23-Pencocokan-string-\(2025\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/23-Pencocokan-string-(2025).pdf)
- Khodra, M. L. (2025). String matching dengan regular expression [Course slides]. Program Studi Teknik Informatika STEI-ITB. [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/24-String-Matching-dengan-Regex-\(2025\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/24-String-Matching-dengan-Regex-(2025).pdf)
- Knuth, D. E., Morris, J. H., Jr, & Pratt, V. R. (1977). Fast Pattern Matching in Strings. SIAM Journal on Computing, 6(2), 323–350. doi:10.1137/0206024
- Boyer, R. S., & Moore, J. S. (1977). A fast string searching algorithm. Commun. ACM, 20(10), 762–772. doi:10.1145/359842.359859
- Levenshtein, V. (1965). Binary codes capable of correcting deletions, insertions, and reversals. Doklady Akademii Nauk SSSR, 163(4), 845-848.
- Aho, A. V., & Corasick, M. J. (1975). Efficient string matching: an aid to bibliographic search. Commun. ACM, 18(6), 333–340. doi:10.1145/360825.360855